A parallel finite element solver for parabolic equations on quantum graphs

Executive summary

Yosef Ben Chedly
Advanced Methods for Scientific Computing, MSc in High Performance Computing Engineering
Politecnico di Milano, a. y. 2025/2026, Supervisor: Prof. Paola F. Antonietti

Abstract

This work develops and validates a finite element framework for the numerical solution of differential problems on metric graphs, with particular emphasis on parabolic equations. The implementation is first verified on elliptic, parabolic and spectral benchmark problems with known reference solutions. The framework is then applied to a Fisher–Kolmogorov diffusion-reaction model for the propagation of misfolded proteins in the brain connectome.
A performance study is subsequently carried out on increasingly refined connectome discretisations. The analysis shows how finite element matrix reordering and parallel programming techniques can be combined to reduce the computational cost of the solver. Exploiting the natural separation between connection-interior unknowns and graph vertices leads to a condensed formulation in which the interior degrees of freedom can be processed independently, while the global problem is reduced to the original graph vertices. The resulting independent per-connection computations are then parallelized using OpenMP, MPI and a hybrid approach. This combination of algebraic reorganisation and parallel execution substantially accelerates the scientific workload while exposing the natural support of both shared- and distributed-memory parallelism.

Introduction

Many physical and biological processes evolve on structures that are naturally described as networks. In a purely combinatorial graph, however, connections only describe which vertices are linked, without resolving the physical behaviour occurring between them. Metric graphs enrich this representation by associating each connection with a one-dimensional domain, so that differential equations can be posed and solved directly along the edges. Quantum graphs further equip this structure with differential operators and appropriate coupling conditions at the vertices.
The first objective of this work is to develop a finite element framework for differential problems posed on such graphs. Each connection is discretised with standard one-dimensional piecewise linear finite elements, while continuity and Neumann–Kirchhoff conditions couple the local edge problems at the graph vertices. The implementation is validated through a sequence of elliptic, parabolic and spectral benchmark problems, including spatial and temporal convergence tests.
The second objective is to investigate the behaviour of the framework on a realistic network-based application. For this purpose, the Fisher–Kolmogorov equation is considered on a brain connectome to describe the diffusion and local replication of misfolded proteins associated with neurodegenerative diseases.
The final objective concerns computational efficiency. Refining the connections increases the number of finite element unknowns while leaving the number of original graph vertices unchanged. This structure is exploited in the performance analysis. First, the effect of sparse matrix ordering and factorization on the sequential solver is investigated. The natural ordering of the unknowns introduced in this work reveals a block structure in which the interior unknowns of different connections are independent. Static condensation (or Schur complement procedure) eliminates these interior unknowns locally, reducing the global problem to a small interface system involving only the original vertices. Shared-memory parallelism is implemented with OpenMP by distributing

connections among threads, while MPI partitions the same work among processes with separate memory spaces. A hybrid MPI-OpenMP implementation combines the two approaches.

Quantum graphs and their discretisation (chapters 1-2)

A quantum graph is a collection of one-dimensional intervals connected at their endpoints to form a network. The term *quantum* refers to the fact that the graph is equipped with a Hamiltonian operator that acts on the functions supported in these intervals, coupled by suitable boundary conditions at the vertices. Two nodes, in particular, are either linked together or not. One can choose a measure for the length of such connections. A combinatorial graph endowed with geometry is a metric graph.
The interior nodes along one edge are numbered consecutively. After all such nodes have been listed for every edge, the original vertices of the graph are numbered last. In this way, the vertices corresponding to the subdivided edges (the subdomain nodes) appear in contiguous blocks within linear algebra structures, while the original graph vertices (the separators) are grouped together at the end of these.

Node numbering: the star-graph figure of Section 1.2 of the full report.

Discretising in time by the θ-method with a uniform Δt, each step of the parabolic solver requires the solution of one linear system,

$$M_\theta\,\mathbf{u}^n = \big(M - (1-\theta)\Delta t\,H\big)\mathbf{u}^{n-1} + \Delta t\big(\theta\,\mathbf{f}^n + (1-\theta)\,\mathbf{f}^{n-1}\big), \qquad M_\theta = M + \theta\,\Delta t\,H.$$

At each simple junction vertex the solution is continuous and the total flux is conserved.

Numerical benchmarks (Chapter 3)

This section presents the numerical benchmarks developed to validate the C++ implementation described in Chapter 7. The tests are organized as a progressive validation path, moving from basic elliptic checks to genuinely parabolic phenomena. We begin with three elliptic test cases, denoted E1–E3, which serve as reference benchmarks for the parabolic extension. Test E1 verifies the correctness of the assembly procedure and the basic control flow by prescribing a constant exact solution. Test E2 assesses the enforcement of mixed Dirichlet–Neumann–Kirchhoff boundary conditions through a linear exact solution. Test E3 investigates the spatial convergence.

Figures 1–5 of the full report (`images/star_constant.png`, `star_linear.png`, `star_sine.png`, `space_convergence.pdf`, `star_radial_decay.png`); temporal convergence in `images/time_convergence.png` (Figure 6).

Finally, three different graph topologies are analyzed in order to compare our parabolic numerical results with the reference ones [6].

Eigenmodes and heat decays: Figures 7–15 of the full report (`images/star_eigenmodes_00_05.png`, `graphene_eigenmodes_*.png`, `tree_eigenmodes_*.png`, `graphene_eigenmode_*.png`, `tree_eigenmode_*.png`); spectral comparison in `images/spectral_comparison.png` (Figure 16).

Spreading of neurodegenerative diseases within the brain's connectome (Chapter 4)

We now consider a more application-oriented test case. In particular, we study the Fisher–Kolmogorov equation on a brain connectome's topology. This problem provides a natural extension

of the previous linear parabolic models, as it combines diffusion along the graph with a nonlinear reaction term. The goal of this section is therefore not to provide a detailed biological study, but to investigate how the developed solver can be extended from controlled benchmark problems to a realistic network-based diffusion-reaction model.
We solve the Fisher-Kolmogorov equation posed on a (metric) graph, like the ones presented in the previous sections; on every edge e:

$$\partial_t c(s,t) - \partial_s\big[D_e\,\partial_s c(s,t)\big] = \alpha(s)\,c(s,t)\,\big[1 - c(s,t)\big].$$

Up to the nonlinear right-hand side, this is the same parabolic problem studied in the first part of this work. The only new ingredient is the nonlinear reaction term.
The graph is extracted from the public *Budapest Reference Connectome v3.0*, which aggregates the tractography of 418 healthy subjects of the *Human Connectome Project*. The downloaded fine-resolution graph contains 1015 parcels and 71604 connections. For each fine connection, the file records how many subjects exhibit that connection, together with aggregate statistics about its fiber count and length. As a first preprocessing step, we retain only fine connections occurring in at least five subjects. This reduces the number of edges from 71604 to 37477, while leaving all 1015 parcels in the graph. The five-subject threshold is inferred from the public dataset as the unique integer threshold reproducing the 37477 fine edges reported in the reference [9]. For comparison, thresholds of four and six subjects yield 40895 and 34718 edges, respectively. The resulting graph is then mapped onto the 83 anatomical regions of the *FreeSurfer* parcellation. Several fine parcels therefore collapse onto the same anatomical region. Among the 37477 retained fine connections, 7895 connect parcels belonging to the same final region and consequently become internal to an aggregated node. The remaining 29582 connections join distinct regions, but many connect different fine parcels belonging to the same pair of anatomical regions; so, such parallel connections are merged, resulting in the final graph with 83 vertices and 1130 edges.

The computational domain: Figures 19–22 of the full report (`images/connectome_views.pdf`, `connectome_regions.pdf`, `connectome_connectogram.pdf`, `connectome_topology.pdf`).

The two hallmark proteins of Alzheimer's disease are *tau* and *amyloid-β*. Amyloid-β is not studied here: the connectome encodes axonal pathways, which are the propagation routes of intracellular tau, whereas amyloid-β spreads extracellularly, which is why the reference itself simulates only tau on this graph.
We reproduce the baseline simulation of [9]: conversion rate $\alpha = 0.5$, misfolded protein seeded in the entorhinal cortex at $c_0 = 0.1$, all other regions healthy and 100 implicit steps of $\Delta t = 0.4$ years. Following the reference, the evolution is summarized by a biomarker, evaluated for the four cortical lobes and for the network as a whole. The simulation is run both with the nodal model, the formulation of the reference implemented literally and with the finite element model, here at one element per connection, so that the two systems share the same 83 unknowns and the same diffusion matrix and differ in the formulation alone. The biomarker curves have the expected sigmoid shape in both. The 50-percent crossing of the network average occurs at 11.11 years for the nodal model and at 12.68 years for the finite element model. Despite reproducing the correct qualitative shape, our model does not recover the separation between the lobe-wise curves observed in the reference results. This separation is particularly important for the clinical reconstruction of disease progression, since different brain regions are expected to become affected at different stages and to follow distinct temporal dynamics. In contrast, our model predicts an almost synchronized evolution across the brain, with the different regions exhibiting essentially the same propagation mechanism and a common temporal dynamics. This behavior is not consistent with the regional heterogeneity observed in PET data and reproduced by the FEM model in [7].

Figure 23 of the full report (`images/biomarker_comparison.pdf`).

For this reason, we observe that the two processes of diffusion and reaction compete and their relative time scales define the Damköhler number

$$\mathrm{Da} = \frac{\tau_{\text{diffusion}}}{\tau_{\text{reaction}}} = \frac{\alpha}{\rho\,\lambda_2},$$

where we find α, the reaction rate, ρ as a coefficient that scales diffusion and λ_2, the smallest non-zero eigenvalue governing the slowest diffusive mode of the network. However, the corresponding eigenvector does not describe a contrast between anatomical lobes, but mainly, in our case, an inter-hemispheric peripheral imbalance. Since our biomarker is defined at the lobe level, we therefore introduce a lobe-scale Damköhler number

$$\mathrm{Da}_{\text{lobe}} = \frac{\alpha}{\rho\,\lambda_{\text{lobe}}},$$

where λ_{lobe} is the slowest relaxation rate associated with concentration differences between lobes. This provides a more appropriate measure of the competition between reaction and diffusion for interpreting the separation of the lobe-wise biomarker curves.
The working hypothesis of this study is that the synchronized lobe-wise curves arise because diffusion, which tends to homogenize the concentration across the network, dominates over reaction. Our objective is therefore to identify a Damköhler number consistent with the separation observed in the results of [9]. Figures 24 and 25 quantitatively support this interpretation: the separation between the curves becomes appreciable as $\mathrm{Da}_{\text{lobe}}$ approaches and exceeds one. At $\rho = 1$, $\mathrm{Da}_{\text{lobe}}$ is 0.05 for the nodal model, while it increases to 0.45 and 0.58 for the finite element model with the consistent and lumped mass matrices, respectively. As discussed in the stability analysis, in fact, mass lumping was adopted as a stabilization technique in the finite element model. In particular, we noticed that the instability develops in two stages: the consistent mass discretization first generates an undershoot and the nonlinear reaction then amplifies it. The nodal model doesn't show instabilities due to the absence of the mass matrix. However, in this first experiment, we have for all the models a $\mathrm{Da}_{\text{lobe}} < 1$, so at the lobe-scale diffusion beats reaction and the synchronized curves are exactly what this number predicts.

Figures 24–25 of the full report (`images/diffusion_scaling.pdf`, `images/lobe_connectivity.pdf`).

The solution to this problem is the finite element model with lumped mass and the fully implicit temporal scheme, at $\rho = 0.005$, corresponding to $\mathrm{Da} = 129$ and $\mathrm{Da}_{\text{lobe}} = 116$. This diffusion scaling is essential for the progression to become visible. We say again that at $\rho = 1$, at the scale used in the references, transport is sufficiently fast to synchronize the network: in a control simulation the four lobes cross the 50% level within 0.03 years of each other and the three stages would appear almost spatially uniform. The choice $\rho = 0.005$ should therefore be interpreted as a calibration of the diffusion time scale against the reference progression, rather than as an additional biological input. Moreover, the introduction of ρ into the model is also motivated by dimensional consistency, as discussed in Section 4.5.
The model reproduces one step of the expected disease sequence: the propagation starts from the entorhinal seed, which is the only tau-specific feature explicitly incorporated into the model and the pathology reaches the temporal lobe first, consistently with the clinical description. However, it does not reproduce the subsequent stages of the expected progression. The remaining lobes are recruited according to the connectivity structure of the graph, with the occipital lobe affected first, followed by the parietal and finally the frontal lobe. This ordering differs from the clinically observed tau propagation sequence, in which the expected progression is temporal $\rightarrow$ frontal $\rightarrow$ parietal $\rightarrow$ occipital. This motivates the introduction of additional biological heterogeneity, such as region-dependent conversion rates, as considered in Section 4.7, where spatial variability is incorporated directly into the reaction term. A further working assumption is that, while biological

information is already intrinsically encoded in the graph and therefore enters the diffusion process through the brain connectivity, additional biological heterogeneity must be introduced into the reaction term, which otherwise remains spatially homogeneous.

Staging against the clinical drawings: Figure 26 of the full report (`images/seeding_patterns_expected.pdf`).

The reaction coefficient is therefore taken piecewise constant over seven anatomical groups, using the mean values reported in Table 1 of [8] (from which we take also the semi-implicit temporal scheme):

$$\alpha_{\text{frontal}} = 0.1801, \quad \alpha_{\text{temporal}} = 0.1421, \quad \alpha_{\text{limbic}} = 0.1351, \quad \alpha_{\text{parietal}} = 0.0627,$$

$$\alpha_{\text{insular}} = 0.1005, \quad \alpha_{\text{occipital}} = 0.0545, \quad \alpha_{\text{subcortical}} = 0.1147.$$

Figure 27 shows the outcome. The tau lobes now cross the 50% level at 14.7, 19.9, 29.7 and 32.4 years, in the order temporal, frontal, parietal and occipital, which is the clinical sequence the uniform rate could not produce. At the parameter values considered in our model, the comparison suggests that the heterogeneous reaction term is the main driver of the regional ordering, while connectivity contributes by constraining the transport pathways and modulating the resulting sequence.

Figure 27 of the full report (`images/seeding_patterns_regional_expected.pdf`).

Performance analysis (Chapter 5)

The performance analysis starts from an unexpected behaviour of the sequential solver. Although almost the entire execution time is spent inside the time loop, the cost per step does not increase monotonically with the number of degrees of freedom. In particular, the mesh with two elements per connection (1213 unknowns) is slower than the one with four elements per connection (3473 unknowns), as shown in Table 6 and Figure 28 of the full report.

Elements per connection	Unknowns	Matrix nonzeros	100 steps [s]	Per step [s]
1	83	2343	0.0832	0.00083
2	1213	5733	1.8694	0.01869
4	3473	12513	1.5486	0.01549
8	7993	26073	2.2343	0.02234

(Table 6 of the full report.)

Figure 28 of the full report (`images/sequential_performance.pdf`).

Since each semi-implicit time step requires the factorization and solution of a sparse linear system, we investigated whether this anomaly was caused by the interaction between matrix ordering and sparse factorization. Different combinations of ordering and factorization were therefore tested, including COLAMD, the natural ordering introduced in Section 1.2, RCM and different LU and LDLT factorizations (Table 7 and Figure 29 of the full report). The experiments show that the anomalous cost of the two-element mesh is caused by fill-in: with the default COLAMD ordering, this smaller problem produces more nonzeros in the factors than the larger four-element mesh. The best configuration among those tested is obtained by combining the natural ordering with LDLT factorization. At 64 elements per connection, the cost of a single time-step decreases from 140.3 ms (COLAMD + SparseLU) to 21.0 ms per step (natural + LDLT).

Table 7 and Figure 29 of the full report (`images/performance_ordering.pdf`).

The effectiveness of the natural ordering follows directly from the structure of the finite element discretization. Interior unknowns are numbered consecutively connection by connection, while the 83 original graph vertices are grouped at the end. The interior part of the matrix is therefore composed of independent tridiagonal blocks, coupled only through the original vertices. This observation motivates a further algorithmic optimization: static condensation.
The interior unknowns of each connection can be eliminated independently, reducing the global problem to a Schur complement system, as discussed in the report.
Rather than assembling and factorizing the complete global sparse matrix, from which one could retrieve the Schur complement, the condensed formulation constructs the reduced 83×83 interface problem directly from the local contributions of the connections. The interior solution is then recovered independently by back substitution.

Elements per connection	Unknowns	Full-system step [ms]	Condensed step [ms]	Ratio
8	7993	3.61	1.03	3.5
16	17033	8.34	1.99	4.2
32	35113	13.52	4.11	3.3
64	71273	35.02	6.73	5.2
128	143593	64.45	10.99	5.9

(Table 8 of the full report.)

Static condensation is between 3.5 and 5.9 times faster than the optimized full-system solver over the tested refinements. At the finest mesh, 10.56 ms of the 10.99 ms step are spent processing the connections, whereas only 0.09 ms are required by the global 83-vertex interface solve. The computational workload is therefore almost entirely composed of independent per-connection operations, naturally exposing parallelism.

Figure 30 of the full report (`images/performance_condensation.pdf`).

After these algorithmic optimizations, we exploit this parallel structure using OpenMP and MPI. In the OpenMP implementation, the connections are distributed among shared-memory threads, each of which independently performs local assembly and condensation. In the MPI implementation, the same work is distributed among separate processes, while the local contributions to the interface problem are combined through `MPI_Allreduce`. Hybrid MPI–OpenMP configurations were also investigated.
The OpenMP strong-scaling experiment is performed on the finest mesh, with 128 elements per connection and 143593 unknowns, on a processor with four physical cores and two hardware threads per core.

Threads	Per step [ms]	Speedup	Efficiency
sequential	9.58	–	–
2	5.65	1.69	0.85
4	3.37	2.84	0.71
8	2.06	4.64	0.58

(Table 9 of the full report.)

The solver reaches a speedup of 2.84 on the four physical cores and 4.64 with eight hardware threads. The efficiency decreases when hyperthreading is used, as the additional threads share the same physical resources. The comparison with MPI and hybrid configurations further shows that, on the tested single shared-memory node, increasing the number of MPI ranks introduces additional synchronization costs without increasing the amount of local numerical work. Pure

OpenMP is therefore the most efficient configuration on this machine, while distributed-memory MPI remains the natural extension for future multi-node experiments.

Table 10 and Figure 31 of the full report (`images/performance_scaling.pdf`).

Conclusions and future work (Chapter 6)

This project developed a finite element framework for partial differential equations posed on metric graphs and applied it to a problem of current interest: the Fisher–Kolmogorov model of neurodegeneration on the brain connectome. The library is built around a compact core consisting of a graph topology with metric connections, a piecewise linear discretization along each connection and a common workflow for input handling, finite element assembly, solution and output.
The performance analysis of Section 5 showed that the unexpected cost of the sequential baseline was due to the fill-in generated by the default matrix ordering during the factorization stage. The natural numbering of the unknowns, with connection interiors first and graph vertices last, exposed a structure that was exploited through static condensation. By eliminating the interior unknowns locally, the global problem was reduced to the 83 original graph vertices. On a four-core laptop, combining this reduction with parallel execution over the interior nodes reduced significantly the per-step cost within the time loop of the finest connectome discretization. Several directions remain open:

- The MPI version was measured only on a single shared-memory node, where the OpenMP implementation is preferable. Since the main purpose of MPI is to enable distributed-memory execution, a multi-node scalability study is the natural next step.
- A per-step cost of about two milliseconds at 143593 unknowns makes many-run studies practical; parameter calibration and the uncertainty quantification of [8] are the settings in which the speedup would be most valuable.
- The data pipeline already reconstructs the fine graph with 1015 nodes from which the 83 regions are aggregated; repeating the study at the finer parcellation would test the robustness of the staging results.
- More advanced disease models from the literature, including multiple protein species and patient-specific parameters, as well as more complex brain-graph domains, could also be incorporated within this same computational framework.

C++ implementation (Chapter 7)

The project is delivered as a single repository organized as follows:

```
FEMG/            the library: headers and sources of the problem classes
test_problem/    one driver per experiment family (parabolic, spectral,
                 fisher_kolmogorov, performance)
benchmarks/      one numbered record per experiment of the report, each
                 with a README, the commands used and the stored results
scripts/         benchmark runners, figure generation and verification
data/            graph files and the connectome preparation pipeline
verification/    internal correctness checks of the solver
report/          this document
```

The records are the organising principle: every figure and every number of the report is produced from files stored under `benchmarks/` and each record documents how to regenerate them.
The library is written in C++17 and built with CMake. It depends on Eigen for sparse and dense

linear algebra and on the Boost Graph Library for the graph topology management. OpenMP and MPI are optional dependencies: when available, CMake enables the parallel performance executables, while the core library can be built and used without either of them. Post-processing uses Python 3 with NumPy, Matplotlib and pandas. The default build type is Release with `-O2`.

Code listings: Sections 7.10–7.11 of the full report (sequential usage example, OpenMP and MPI excerpts).

The used references can be consulted in the bibliography of the full version of this work.